**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ**
**ІМЕНІ ІВАНА ФРАНКА**

# ЗВІТ
# ДО ЛАБОРАТОРНОЇ РОБОТИ №6

## «Автоматизоване тестування Python-програм»

з курсу «Професійний Python»
Варіант №3 — «Інтернет-магазин»

Виконав(ла): Вересюк Ірина
Група: ФЕП-32
Перевірив(ла): Куньо І.М.
Дата: 01.11.26

Львів — 2026

## 1. Мета роботи

Сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest: проєктувати unit та integration tests, створювати fixtures, застосовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності, використовувати Mock і AsyncMock, а також аналізувати statement і branch coverage за допомогою pytest-cov.

## 2. Індивідуальний варіант і опис тестованого проєкту

Варіант №3 — «Інтернет-магазин». Проєкт shop_project реалізує каталог товарів з такими можливостями: створення Product із валідацією price/quantity; додавання товару до каталогу; пошук за назвою та SKU; фільтрація за category; розрахунок вартості складських залишків; визначення найдорожчого товару; визначення товарів із низьким запасом (мінімальний stock береться з конфігурації); CSV import; JSON export; розрахунок знижки через зовнішній DiscountProvider; сповіщення про низький запас через Notifier; асинхронне оновлення цін через зовнішній PriceProvider.

Результатом роботи є test suite із 72 тестів (65 unit та 7 integration), який запускається однією командою та формує звіт про coverage (statement + branch).

## 3. Структура production code

```
shop_project/
├── pyproject.toml
├── README.md
├── .gitignore
├── src/
│   └── shop/
│       ├── __init__.py
│       ├── exceptions.py     # власні exceptions
│       ├── models.py         # Product + валідація
│       ├── protocols.py      # інтерфейси залежностей
│       ├── repositories.py  # InMemoryProductRepository
│       ├── config.py         # Config, SHOP_MIN_STOCK
│       ├── services.py       # Catalog, PricingService, StockMonitor
│       ├── importers.py      # CSV import
│       ├── exporters.py      # JSON export
│       ├── providers.py      # async PriceProvider
│       ├── pipeline.py       # import → processing → export
│       └── main.py           # точка входу
└── tests/
    ├── conftest.py
    ├── unit/
    │   ├── test_models.py
    │   ├── test_catalog.py
    │   ├── test_config.py
    │   ├── test_importers_exporters.py
    │   ├── test_repository.py
    │   └── test_services_mocks.py
    └── integration/
        └── test_pipeline.py
```

Модуль models.py містить dataclass Product та валідацію; services.py — business logic (Catalog, PricingService, StockMonitor); залежності (repository, notifier, discount/price providers) описано через Protocol у protocols.py, що дає змогу легко підміняти їх Mock-ами. Власні exceptions: ProductValidationError, MalformedRecordError, DuplicateProductError, ProductNotFoundError, ConfigurationError, ExternalServiceError.

### 3.1. Фрагмент models.py

```
@dataclass(slots=True)
class Product:
    """A product in the online shop catalog."""

    sku: str
    name: str
    category: str
    price: float
    quantity: int

    def __post_init__(self) -> None:
        self.sku = self._require_text(self.sku, "sku")
        self.name = self._require_text(self.name, "name")
        self.category = self._require_text(self.category, "category")
        if isinstance(self.price, bool) or not isinstance(self.price, (int, float)):
            raise ProductValidationError("Price must be a number")
        if self.price < 0:
            raise ProductValidationError("Price cannot be negative")
        if isinstance(self.quantity, bool) or not isinstance(self.quantity, int):
            raise ProductValidationError("Quantity must be an integer")
        if self.quantity < 0:
            raise ProductValidationError("Quantity cannot be negative")

    @staticmethod
    def _require_text(value: str, field: str) -> str:
        if not isinstance(value, str) or not value.strip():
            raise ProductValidationError(f"Field '{field}' must not be empty")
        return value.strip()

    @property
    def stock_value(self) -> float:
        """Total value of the product stock."""
        return self.price * self.quantity
```

## 4. Структура test suite

| Файл | Тестів | Що перевіряє |
|---|---|---|
| tests/unit/test_models.py | 17 | Product: створення, validation, boundary, invalid |
| tests/unit/test_catalog.py | 13 | Catalog: додавання, пошук, фільтр, stock value, low stock |
| tests/unit/test_config.py | 6 | Config через monkeypatch (env SHOP_MIN_STOCK) |
| tests/unit/test_importers_exporters.py | 8 | CSV import, malformed records, JSON export (tmp_path) |
| tests/unit/test_repository.py | 3 | InMemoryProductRepository |

| Файл | Тестів | Що перевіряє |
|---|---|---|
| tests/unit/test_services_mocks.py | 18 | Mock/AsyncMock: repository, pricing, notifier, providers |
| tests/integration/test_pipeline.py | 7 | Повні сценарії import → processing → export, main() |
| Разом | 72 | 65 unit + 7 integration |

Integration tests позначено маркером @pytest.mark.integration (pytestmark у модулі), маркер зареєстровано в pyproject.toml. Тому можна запускати окремо: pytest -m "not integration" (65 тестів) або pytest -m integration (7 тестів).

## 5. Test matrix

| Поведінка | Тип | Тести |
|---|---|---|
| Створення Product | normal | test_product_creation_strips_text_fields, test_to_dict_contains_all_fields |
| Валідація price/quantity | boundary | test_boundary_price_and_quantity_are_accepted[…] (0, 0.0, великі значення) |
| Валідація price/quantity | invalid | test_invalid_price_or_quantity_raises_validation_error[…] (6 випадків) |
| Порожні sku/name/category | invalid | test_empty_text_field_is_rejected[…] |
| Додавання товару | normal / failure | test_add_product_makes_it_findable, test_duplicate_sku_is_rejected |
| Пошук товару | normal / failure | test_find_unknown_product_raises_not_found, test_search_by_name_is_case_insensitive |
| Фільтрація за category | normal / empty | test_filter_by_category, test_filter_by_unknown_category_returns_empty_list |
| Вартість залишків, найдорожчий | normal / empty | test_total_stock_value, test_most_expensive_product, test_empty_catalog_… |
| Мінімальний stock (config) | boundary | test_low_stock_boundary[4 / 5 / 6] |
| Configuration | normal / invalid | test_config.py (monkeypatch.setenv / delenv) |
| CSV import | normal / invalid | test_csv_import_reads_all_products, test_malformed_record_raises_with_line_number |
| JSON export | normal / empty | test_json_export_writes_valid_file, test_json_export_of_empty_list |
| ProductRepository (Mock) | interaction | test_catalog_find_uses_repository_get, test_add_product_delegates_to_repository |
| Pricing (return_value/side_effect) | normal / failure | test_discount_applied_…, test_provider_failure_is_wrapped_via_side_effect |
| Notifier (interaction) | interaction | test_stock_monitor_notifies_…, …_does_not_notify_…, test_notifier_failure_propagates |
| Async provider (AsyncMock) | async | test_refresh_prices_with_async_mock, test_refresh_prices_propagates_provider_failure |
| Pipeline | integration | test_pipeline.py (7 сценаріїв) |

## 6. Приклади unit tests

```
def test_product_creation_strips_text_fields():
    product = Product("  A-1 ", " Laptop ", " electronics ", 100, 2)
    assert (product.sku, product.name, product.category) == ("A-1", "Laptop", "electronics")
```

```
def test_search_by_name_is_case_insensitive(catalog):
    names = [p.name for p in catalog.search_by_name("  MOU ")]
    assert names == ["Mouse"]
```

```
def test_empty_catalog_has_zero_value_and_no_best_product(empty_catalog):
    assert empty_catalog.total_stock_value() == 0
    assert empty_catalog.most_expensive() is None
    assert empty_catalog.all_products() == []
```

## 7. Приклад integration test

Тест створює тимчасовий CSV, запускає повний pipeline (CSV reader → validation → Catalog → JSON exporter) і перевіряє вихідний JSON та статистику.

```
def test_import_processing_export_pipeline(csv_file, tmp_path):
    out = tmp_path / "result.json"
    stats = run_pipeline(csv_file, out, Config(min_stock=5))
    exported = json.loads(out.read_text(encoding="utf-8"))
    assert [p["sku"] for p in exported] == ["A-1", "B-2", "C-3"]
    assert stats["total_stock_value"] == pytest.approx(89050.0)
    assert stats["low_stock_count"] == 1
    assert stats["most_expensive_price"] == 25000
```

## 8. Fixtures

### 8.1. Приклад fixture (conftest.py)

```
@pytest.fixture
def laptop() -> Product:
    return Product("A-1", "Laptop", "electronics", 25000.0, 3)



@pytest.fixture
def catalog(sample_products: list[Product]) -> Catalog:
    catalog = Catalog(InMemoryProductRepository(), Config(min_stock=5))
    for product in sample_products:
        catalog.add_product(product)
    return catalog
```

### 8.2. Fixture із yield (setup / teardown)

```
@pytest.fixture
def csv_file(tmp_path):
    """Temporary CSV file; removed message printed in teardown via yield."""
    path = tmp_path / "products.csv"
```

```
    path.write_text(
        "sku,name,category,price,quantity\n"
        "A-1,Laptop,electronics,25000,3\n"
        "B-2,Mouse,electronics,500,20\n"
        "C-3,Notebook,stationery,40.5,100\n",
        encoding="utf-8",
    )
    yield path
    path.unlink(missing_ok=True)
```

Fixtures laptop, sample_products, catalog, empty_catalog, csv_file винесені в conftest.py і доступні всім тестам без імпорту. Усі fixtures мають default scope function, тому кожен тест працює з власним незалежним станом.

## 9. Parametrization

```
@pytest.mark.parametrize(
    "price, quantity, message",
    [
        (-0.01, 1, "Price cannot be negative"),
        (10, -1, "Quantity cannot be negative"),
        ("10", 1, "Price must be a number"),
        (True, 1, "Price must be a number"),
        (10, 1.5, "Quantity must be an integer"),
        (10, True, "Quantity must be an integer"),
    ],
)
def test_invalid_price_or_quantity_raises_validation_error(price, quantity, message):
    with pytest.raises(ProductValidationError, match=message):
        Product("X", "Item", "misc", price, quantity)
```

Параметризовані тести створюють окремий результат для кожного набору даних; ids роблять назви зрозумілими (наприклад, test_low_stock_boundary[at-threshold]).

## 10. Boundary test

Мінімальний stock = 5. Товар із quantity 4 вважається low stock, із 5 та 6 — ні:

```
@pytest.mark.parametrize(
    "quantity, expected_low",
    [(4, True), (5, False), (6, False)],
    ids=["below-threshold", "at-threshold", "above-threshold"],
)
def test_low_stock_boundary(empty_catalog, quantity, expected_low):
    empty_catalog.add_product(Product("X", "Item", "misc", 1, quantity))
    assert bool(empty_catalog.low_stock_products()) is expected_low
```

## 11. pytest.raises

```
def test_find_unknown_product_raises_not_found(catalog):
    with pytest.raises(ProductNotFoundError, match="Z-9"):
        catalog.find_product("Z-9")
```

```
def test_empty_text_field_is_rejected(field):
    data = {"sku": "X", "name": "Item", "category": "misc", "price": 1, "quantity": 1}
```

```
    data[field] = "   "
    with pytest.raises(ProductValidationError) as exc_info:
        Product(**data)
    assert field in str(exc_info.value)
```

Другий приклад використовує exc_info для перевірки повідомлення, а також перевіряється custom exception ProductValidationError (він наслідує ValueError, що перевіряє test_custom_exception_is_also_value_error).

## 12. pytest.approx

```
def test_stock_value_uses_approx_for_floats():
    product = Product("X", "Pen", "stationery", 0.1, 3)
    assert product.stock_value == pytest.approx(0.3)
```

0.1 * 3 у типових float-обчисленнях не дорівнює точно 0.3, тому для порівняння використано pytest.approx.

## 13. tmp_path

```
def test_json_export_writes_valid_file(tmp_path, sample_products):
    target = tmp_path / "out.json"
    count = export_products_to_json(sample_products, target)
    data = json.loads(target.read_text(encoding="utf-8"))
    assert count == 3
```

Тимчасові файли створюються у каталозі tmp_path, який pytest створює для кожного тесту й автоматично очищує; реальна файлова система проєкту не зачіпається.

## 14. monkeypatch

```
def test_min_stock_is_read_from_environment(monkeypatch):
    monkeypatch.setenv(ENV_MIN_STOCK, "12")
    assert Config.from_env().min_stock == 12
```

```
def test_default_config_when_env_missing(monkeypatch):
    monkeypatch.delenv(ENV_MIN_STOCK, raising=False)
    assert Config.from_env().min_stock == DEFAULT_MIN_STOCK
```

Змінна середовища SHOP_MIN_STOCK задається або видаляється лише на час тесту; після його завершення значення відновлюється автоматично.

## 15. Mock, return_value, side_effect, interaction assertions

### 15.1. Mock репозиторію + return_value

```
def test_catalog_find_uses_repository_get(laptop):
    repo = Mock()
    repo.get.return_value = laptop
    assert Catalog(repo).find_product("A-1") is laptop
    repo.get.assert_called_once_with("A-1")
```

### 15.2. Mock зі spec та return_value (pricing)

```
def test_discount_applied_from_provider_return_value(laptop):
    provider = Mock(spec=DiscountProvider)
    provider.get_discount.return_value = 10
    assert PricingService(provider).discounted_price(laptop) == pytest.approx(22500.0)
    provider.get_discount.assert_called_once_with("electronics")
```

### 15.3. side_effect (failure path зовнішньої залежності)

```
def test_provider_failure_is_wrapped_via_side_effect(laptop):
    provider = Mock(spec=DiscountProvider)
    provider.get_discount.side_effect = ConnectionError("timeout")
    with pytest.raises(ExternalServiceError, match="failed") as exc_info:
        PricingService(provider).discounted_price(laptop)
    assert isinstance(exc_info.value.__cause__, ConnectionError)
```

### 15.4. Interaction assertions (Notifier)

```
def test_stock_monitor_notifies_for_each_low_stock_product(catalog):
    notifier = Mock(spec=Notifier)
    assert StockMonitor(catalog, notifier).check() == 1
    notifier.notify.assert_called_once_with("Low stock: A-1 (3 left)")
```

```
def test_stock_monitor_does_not_notify_when_stock_is_fine(empty_catalog):
    notifier = Mock(spec=Notifier)
    assert StockMonitor(empty_catalog, notifier).check() == 0
    notifier.notify.assert_not_called()
```

Використано assert_called_once_with та assert_not_called — це interaction testing: перевіряється не стан, а факт і параметри взаємодії з залежністю.

## 16. AsyncMock

У проєкті є асинхронна зовнішня залежність PriceProvider (метод fetch_price), тому AsyncMock застосовано. Для запуску async-тестів використано pytest-asyncio (asyncio_mode = "auto").

```
async def test_refresh_prices_with_async_mock(sample_products):
    provider = AsyncMock()
    provider.fetch_price.side_effect = [26000.0, 500.0, 45.0]
    updated = await refresh_prices(sample_products, provider)
    assert updated == 2
    assert sample_products[0].price == 26000.0
    assert provider.fetch_price.await_count == 3
    provider.fetch_price.assert_any_await("A-1")



async def test_refresh_prices_propagates_provider_failure(laptop):
    provider = AsyncMock()
```

## 17. Конфігурація pytest та coverage (pyproject.toml)

```
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]
asyncio_mode = "auto"
```

```
asyncio_default_fixture_loop_scope = "function"
addopts = ["-ra", "--strict-markers"]
markers = [
    "integration: integration tests",
]

[tool.coverage.run]
branch = true
source = ["shop"]

[tool.coverage.report]
show_missing = true
skip_covered = false
fail_under = 80
```

## 18. Результат запуску tests

Команда запуску:

```
pytest -v --cov=shop --cov-branch --cov-report=term-missing --cov-report=html
```

```
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/shop_project
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0, asyncio-1.4.1
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=function,
asyncio_default_test_loop_scope=function
collecting ... collected 72 items

tests/integration/test_pipeline.py::test_import_processing_export_pipeline PASSED [  1%]
tests/integration/test_pipeline.py::test_import_processing_export_pipeline PASSED [  1%]
tests/integration/test_pipeline.py::test_pipeline_fails_on_malformed_csv_and_writes_nothing PASSED [
2%]
tests/integration/test_pipeline.py::test_pipeline_rejects_duplicate_skus PASSED [  4%]
tests/integration/test_pipeline.py::test_main_uses_min_stock_from_environment PASSED [  5%]
tests/integration/test_pipeline.py::test_main_prints_usage_for_wrong_arguments PASSED [  6%]
tests/integration/test_pipeline.py::test_empty_csv_produces_empty_export PASSED [  8%]
tests/integration/test_pipeline.py::test_async_price_refresh_then_export PASSED [  9%]
tests/unit/test_catalog.py::test_add_product_makes_it_findable PASSED    [ 11%]
tests/unit/test_catalog.py::test_duplicate_sku_is_rejected PASSED        [ 12%]
tests/unit/test_catalog.py::test_find_unknown_product_raises_not_found PASSED [ 13%]
tests/unit/test_catalog.py::test_search_by_name_is_case_insensitive PASSED [ 15%]
tests/unit/test_catalog.py::test_filter_by_category PASSED               [ 16%]
tests/unit/test_catalog.py::test_filter_by_unknown_category_returns_empty_list PASSED [ 18%]
tests/unit/test_catalog.py::test_total_stock_value PASSED                [ 19%]
...
```

| Показник | Значення | Коментар |
|---|---|---|
| Зібрано tests | 72 | 65 unit + 7 integration |
| Passed | 72 | |
| Failed | 0 | після виправлення помилки в очікуваному значенні (див. розділ 22) |
| Час виконання | 0.31 s | tests швидкі й детерміновані |

## 19. Coverage report та branch coverage

```
Name                        Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------
src/shop/__init__.py            1      0      0      0   100%
src/shop/config.py             21      0      4      0   100%
src/shop/exceptions.py          7      0      0      0   100%
src/shop/exporters.py           8      0      0      0   100%
src/shop/importers.py          19      0      4      0   100%
src/shop/main.py               16      1      6      1    91%   23
src/shop/models.py             32      0     10      0   100%
src/shop/pipeline.py           14      0      2      0   100%
src/shop/protocols.py           7      0      0      0   100%
src/shop/providers.py          22      0      4      0   100%
src/shop/repositories.py       14      0      2      0   100%
src/shop/services.py           50      0      6      0   100%
-------------------------------------------------------------------------
TOTAL                         211      1     38      1    99%
Coverage HTML written to dir htmlcov
Required test coverage of 80.0% reached. Total coverage: 99.20%
```

Загальне покриття — 99.20 % (211 statements, 1 пропущений; 38 branches, 1 частково пройдений), що перевищує поріг fail_under = 80. Branch coverage увімкнено опцією branch = true у [tool.coverage.run] та ключем --cov-branch.

## 20. Фрагмент HTML coverage report

HTML-звіт формується в каталозі htmlcov/ (відкривається файл htmlcov/index.html). Фрагмент головної таблиці:

| Module | Statements | Missing | Branches | Partial | Coverage | Branch cov. |
|---|---|---|---|---|---|---|
| src/shop/config.py | 21 | 0 | 4 | 0 | 100% | 100% |
| src/shop/importers.py | 19 | 0 | 4 | 0 | 100% | 100% |
| src/shop/main.py | 16 | 1 | 6 | 1 | 91% | 83% |
| src/shop/models.py | 32 | 0 | 10 | 0 | 100% | 100% |
| src/shop/services.py | 50 | 0 | 6 | 0 | 100% | 100% |
| Total | 211 | 1 | 38 | 1 | 99% | |

## 21. Аналіз недостатньо протестованих paths

Єдиний непокритий рядок — src/shop/main.py:23, тобто виклик raise SystemExit(main()) усередині блоку if __name__ == "__main__":. Цей блок виконується лише при безпосередньому запуску модуля як скрипта, а тести викликають функцію main() напряму. Через це також з’являється один частково пройдений branch (83 % branch coverage для main.py).

Щоб покрити цей шлях, потрібен тест, який запускає модуль як програму, наприклад через runpy.run_module("shop.main", run_name="__main__") із підміненим sys.argv та перевіркою очікуваного SystemExit, або через subprocess.run([sys.executable, "-m", "shop.main"]) із перевіркою коду завершення. Оскільки логіка цього рядка тривіальна (передача коду завершення в

SystemExit), а уся поведінка main() уже перевірена (test_main_uses_min_stock_from_environment, test_main_prints_usage_for_wrong_arguments), я свідомо залишила його непокритим і не додавала тест лише заради відсотка.

## 22. Знайдені тестами дефекти

Реальні production defects у ході виконання роботи не були виявлені. Проте тестування виявило такі проблеми та підтвердило ефективність тестів:

1) Failing test під час першого запуску. Інтеграційний тест test_import_processing_export_pipeline впав: "assert 89050.0 == 79050.0 ± 0.07905". Аналіз показав, що production code правильний (25000·3 + 500·20 + 40.5·100 = 89050), а помилку допущено в очікуваному значенні тесту (арифметична помилка). Очікуване значення виправлено, після цього 72 з 72 тестів пройшли.

2) Навмисне пошкодження production function (демонстрація для захисту). У services.py умову p.quantity < self._config.min_stock було змінено на p.quantity <= self._config.min_stock. Результат запуску:

```
    assert bool(empty_catalog.low_stock_products()) is expected_low
E   AssertionError: assert True is False
FAILED tests/unit/test_catalog.py::test_low_stock_boundary[at-threshold]
1 failed, 71 passed in 0.13s
```

Boundary test на значенні quantity = 5 (на самій межі) негайно виявив помилку. Після повернення оригінального коду всі 72 тести знову проходять. Це показує цінність boundary-тестів: без значення на межі помилка «< замість <=» залишилася б непоміченою.

## 23. Робота з Git

```
git init
git add .gitignore pyproject.toml README.md src/shop/__init__.py src/shop/exceptions.py \
        src/shop/models.py src/shop/protocols.py
git commit -m "Create project structure and domain model"
git add src/shop tests/conftest.py
git commit -m "Add services, repositories, import/export and config"
git add tests
git commit -m "Add unit and integration test suite"
```

### 23.1. Результат git status

```
On branch master
nothing to commit, working tree clean
```

### 23.2. Результат git log --oneline

```
4f63e9a Add unit and integration test suite
64fcd52 Add services, repositories, import/export and config
b183b84 Create project structure and domain model
```

## 24. Відповіді на контрольні запитання

**1. Що таке automated test?**

Програма, яка автоматично перевіряє поведінку іншої програми й повідомляє passed/failed без ручної участі.

**2. Які основні переваги автоматизованого тестування?**

Раннє виявлення defects, захист від regressions, безпечний refactoring, документування очікуваної поведінки та підтримка якості в CI.

**3. Що таке regression?**

Поява помилки в раніше робочій функціональності після внесення змін.

**4. Що таке unit test?**

Тест, що ізольовано перевіряє невелику одиницю поведінки (функцію, клас).

**5. Що таке integration test?**

Тест, що перевіряє взаємодію кількох компонентів разом.

**6. Чим integration test відрізняється від unit test?**

Unit test ізолює одну одиницю й підміняє залежності; integration test використовує реальні компоненти (наприклад, файли) і перевіряє їх спільну роботу.

**7. Що показує test pyramid?**

Співвідношення типів тестів: багато швидких unit tests, менше integration, ще менше дорогих end-to-end.

**8. Чому unit tests зазвичай більше?**

Вони швидкі, дешеві, стабільні й точно локалізують проблему.

**9. Що таке pytest?**

Фреймворк для автоматизованого тестування Python-програм із простим assert, fixtures і плагінами.

**10. Як pytest знаходить tests?**

За шаблоном файлів test_*.py або *_test.py та функцій/методів test_*.

**11. Якою командою запускається pytest?**

pytest (або python -m pytest) у корені проєкту.

**12. Для чого pytest -v?**

Для детального виводу: імена кожного тесту та їх результат.

**13. Для чого pytest -x?**

Щоб зупинити запуск після першого failure.

**14. Що означає Arrange–Act–Assert?**

Структура тесту: підготувати дані, виконати дію, перевірити результат.

**15. Для чого використовується assert?**

Для перевірки умови; якщо вона хибна, тест падає. Pytest показує детальне порівняння значень.

**16. Що таке assertion failure?**

Невиконання assert, тобто фактичний результат не збігся з очікуваним.

**17. Чому floats не завжди слід порівнювати через ==?**

Через похибки подання чисел із плаваючою комою (0.1 * 3 != 0.3).

**18. Для чого pytest.approx?**

Для порівняння чисел із допустимою похибкою.

**19. Для чого pytest.raises?**

Для перевірки, що код викидає очікуваний exception.

**20. Як перевірити exception message?**

Параметром match у pytest.raises(..., match="текст") (regex-пошук у повідомленні).

**21. Як отримати exception object через exc_info?**

with pytest.raises(E) as exc_info: ... ; далі exc_info.value містить об’єкт виключення.

**22. Що таке boundary testing?**

Тестування значень на межах допустимих діапазонів та поруч із ними.

**23. Які boundaries потрібно тестувати для діапазону 0–100?**

-1, 0, 1, 99, 100, 101 (мінімум: межі та значення одразу за межами).

**24. Що таке parametrization?**

Запуск одного тесту з різними наборами вхідних даних.

**25. Для чого pytest.mark.parametrize?**

Щоб описати набори параметрів і не дублювати код однотипних тестів.

**26. Що дають ids параметризованих tests?**

Зрозумілі імена окремих випадків у звіті, що спрощує пошук проблеми.

**27. Що таке fixture?**

Функція з декоратором @pytest.fixture, що готує дані або ресурси для тестів.

**28. Для чого використовуються fixtures?**

Для повторного використання setup-коду, ізоляції та передачі готових об’єктів у тести.

**29. Що означає fixture setup?**

Підготовка ресурсу до виконання тесту (все до yield / return).

**30. Що означає fixture teardown?**

Очищення ресурсів після тесту (код після yield).

**31. Для чого yield у fixture?**

Віддати значення тесту й після завершення тесту виконати teardown.

**32. Які fixture scopes підтримує pytest?**

function, class, module, package, session.

**33. Який scope є default?**

function — fixture створюється заново для кожного тесту.

**34. Для чого conftest.py?**

Для спільних fixtures і hooks, доступних тестам у каталозі та підкаталогах.

**35. Чому fixtures із conftest.py не імпортують вручну?**

Pytest автоматично знаходить conftest.py і підключає fixtures за іменем аргументу.

**36. Що таке tmp_path?**

Вбудована fixture, що надає шлях (pathlib.Path) до унікального тимчасового каталогу.

**37. Чому tmp_path корисний?**

Тести не засмічують проєкт, не залежать один від одного, файли чистяться автоматично.

**38. Що таке test isolation?**

Незалежність тестів: результат одного не залежить від порядку чи побічних ефектів інших.

**39. Для чого monkeypatch?**

Для тимчасової підміни атрибутів, функцій, змінних середовища, словників тощо на час тесту.

**40. Як monkeypatch змінює environment variable?**

monkeypatch.setenv("NAME", "value").

**41. Як видалити environment variable у test?**

monkeypatch.delenv("NAME", raising=False).

**42. Як підмінити function через monkeypatch?**

monkeypatch.setattr(module, "func", fake_func) (або за рядковим шляхом).

**43. Що таке test double?**

Об’єкт-замінник реальної залежності в тесті (Dummy, Stub, Fake, Mock, Spy).

**44. Що таке Dummy?**

Об’єкт, який лише заповнює параметр і не використовується.

**45. Що таке Stub?**

Об’єкт із наперед заданими відповідями на виклики.

**46. Що таке Fake?**

Спрощена робоча реалізація (наприклад, in-memory repository).

**47. Що таке Mock?**

Об’єкт, що записує виклики й дає змогу перевіряти взаємодію (assert_called_*).

**48. Що таке Spy?**

Об’єкт, що виконує реальну поведінку й одночасно записує виклики.

**49. Для чого Mock?**

Ізолювати тестований код від зовнішніх залежностей і перевіряти взаємодію з ними.

**50. Що таке spec?**

Параметр Mock(spec=Class), що обмежує атрибути Mock-а інтерфейсом реального класу й ловить помилки в іменах методів.

**51. Для чого return_value?**

Щоб задати значення, яке повертає Mock під час виклику.

**52. Для чого side_effect?**

Щоб Mock викидав exception, повертав послідовність значень або виконував функцію.

**53. Як перевірити, що method викликано один раз?**

mock.method.assert_called_once() або assert_called_once_with(...).

**54. Як перевірити parameters виклику?**

assert_called_with(...), assert_called_once_with(...) або mock.call_args.

**55. Як перевірити, що method не викликався?**

mock.method.assert_not_called().

**56. Що таке state testing?**

Перевірка кінцевого стану або результату після виконання дії.

**57. Що таке interaction testing?**

Перевірка того, як код взаємодіє із залежностями (які методи й з якими аргументами викликано).

**58. Коли interaction testing виправдане?**

Коли результатом дії є виклик зовнішньої системи (notifier, email, API), який складно перевірити за станом.

**59. Що таке patch?**

Функція/контекстний менеджер unittest.mock.patch, що тимчасово замінює об’єкт на Mock.

**60. Чому patch потрібно виконувати там, де name використовується?**

Бо імпортоване ім’я зберігається в просторі імен модуля-споживача; підміна в місці визначення не вплине на вже імпортоване посилання.

**61. Що таке AsyncMock?**

Різновид Mock для async-функцій: його виклик повертає awaitable.

**62. Як перевірити awaited call?**

assert_awaited(), assert_awaited_once_with(...), assert_any_await(...), await_count.

**63. Що таке flaky test?**

Тест, що іноді проходить, а іноді падає без змін коду.

**64. Чому flaky tests небезпечні?**

Вони знижують довіру до test suite, приховують справжні помилки й гальмують CI.

**65. Що таке deterministic test?**

Тест, що за однакових умов завжди дає однаковий результат.

**66. Чому unit test не повинен залежати від Internet?**

Мережа нестабільна й повільна, що робить тест недетермінованим; залежності підміняють Mock/Fake.

**67. Коли в integration test доцільний real filesystem?**

Коли потрібно перевірити реальну роботу з файлами (import/export), бажано через tmp_path.

**68. Чому не потрібно mock-ати все?**

Надмірні Mock-и перевіряють лише самі Mock-и й не виявляють реальних помилок взаємодії.

**69. Що таке over-mocking?**

Надмірне використання Mock-ів, коли тест прив'язаний до реалізації й ламається при refactoring без зміни поведінки.

**70. Що таке coverage?**

Метрика того, яка частина коду виконується під час тестів.

**71. Що таке statement coverage?**

Частка виконаних рядків (statements) коду.

**72. Що таке branch coverage?**

Частка пройдених гілок умовних переходів (if/else, цикли).

**73. Чим вони відрізняються?**

Statement coverage може бути 100 %, навіть якщо не пройдено else-гілку; branch coverage вимагає пройти обидві гілки кожної умови.

**74. Як отримати coverage через pytest-cov?**

pytest --cov=shop --cov-branch.

**75. Для чого --cov-report=term-missing?**

Щоб у терміналі показати номери рядків, не покритих тестами.

**76. Для чого HTML coverage report?**

Для зручного перегляду коду з підсвіткою покритих і непокритих рядків (--cov-report=html).

**77. Чи гарантує 100% coverage відсутність bugs?**

Ні: coverage показує лише, що код виконувався, а не що результат перевірено правильно.

**78. Чому meaningful assertions важливіші за coverage percentage?**

Тест без змістовних перевірок може давати високе покриття, але не виявляти помилок.

**79. Що таке coverage threshold?**

Мінімальний відсоток покриття (fail_under), нижче якого запуск вважається невдалим.

**80. Чому tests повинні бути незалежними один від одного?**

Щоб їх можна було запускати в будь-якому порядку, окремо й паралельно, а падіння одного не впливало на інші.

## 25. Висновок

У лабораторній роботі створено повноцінний automated test suite для проєкту «Інтернет-магазин» (варіант №3). Реалізовано 72 тести (65 unit та 7 integration), які перевіряють normal cases, boundary cases, invalid input, exception paths, файловий import/export, configuration, взаємодію компонентів і зовнішні залежності. Використано fixtures (зокрема з yield), параметризацію, pytest.raises, pytest.approx, tmp_path, monkeypatch, Mock із return_value та side_effect, interaction assertions та AsyncMock.

Запуск pytest із pytest-cov показав 72 passed, 0 failed та загальне покриття 99.20 % (statement + branch), що перевищує порогове значення 80 %. Аналіз coverage виявив єдиний непокритий рядок — виклик SystemExit у блоці if __name__ == "__main__". Демонстрація навмисного пошкодження

коду підтвердила, що boundary-тест виявляє помилку. Отримано практичні навички проєктування тестів та аналізу coverage.